

chapter 2 Application Layer

计算机网络应用层核心知识点整理

了解各种版本HTTP的特点，了解DNS技术细节，区别C/S和P2P结构，CDN，熟悉各种应用层协议名称，了解socket programming with UDP and TCP

一、HTTP 协议演进

HTTP/1.0

- **特点:** 短连接，每次请求 / 响应都建立和关闭一次 TCP 连接。
- **性能:** 开销大，效率低，存在明显的队头阻塞问题。

HTTP/1.1

- **特点:** 默认使用长连接（Persistent Connection），允许在一个 TCP 连接上发送多个请求。
- **改进:** 引入了管道化（Pipelining）、分块传输编码（Chunked Transfer Encoding）和更强大的缓存机制（如 Cache-Control）。
- **性能:** 相比 1.0 有显著提升，但管道化并未完全解决队头阻塞，且头部信息冗余。

HTTP/2

- **特点:** 基于二进制帧（Binary Framing），实现了真正的多路复用（Multiplexing）。
- **改进:** 头部压缩（HPACK）、服务器推送（Server Push）。
- **性能:** 彻底解决了应用层的队头阻塞，但底层仍基于 TCP，存在 TCP 层的队头阻塞。

HTTP/3

- **特点:** 基于 QUIC 协议，该协议底层使用 UDP。
- **改进:** 实现了 0-RTT 连接建立、内置 TLS 加密、并彻底解决了队头阻塞问题。
- **性能:** 特别适用于高延迟、高丢包的移动网络环境，性能潜力巨大。

二、DNS 技术细节

DNS 的作用

- DNS（Domain Name System）是互联网的“电话簿”，负责将人类易于记忆的域名（如 www.google.com）解析为计算机可识别的 IP 地址。

DNS 工作原理

1. **用户请求:** 用户在浏览器中输入域名。
2. **递归查询:** 本地 DNS 服务器接收到请求后，如果本地缓存没有记录，则会向根服务器发起查询。
3. **迭代查询:** 根服务器返回顶级域（TLD）服务器的地址，本地 DNS 服务器再向 TLD 服务器查询，依此类推，直到查询到目标 IP 地址。
4. **结果返回:** 本地 DNS 服务器将获取到的 IP 地址返回给用户，并将其缓存，以备后续查询使用。
5. 本根顶权

常见 DNS 记录类型

- **A 记录:** 将域名映射到 IPv4 地址。
- **AAAA 记录:** 将域名映射到 IPv6 地址。
- **CNAME 记录:** 将一个域名（别名）指向另一个域名。
- **MX 记录:** 指定负责接收该域名邮件的邮件服务器。
- **NS 记录:** 指定该域名由哪个 DNS 服务器进行解析。

三、C/S 与 P2P 结构区别

C/S (Client/Server) 结构

- **定义:** 客户端 - 服务器模式，网络中存在一个或多个中心服务器，客户端通过网络向服务器请求服务。
- **优点:**
 - 集中管理，易于维护和控制。
 - 安全性较高，数据集中存储。
 - 服务器性能通常较强，能提供稳定服务。
- **缺点:**
 - 服务器容易成为性能瓶颈。
 - 扩展性相对较差，增加用户需要增加服务器资源。
 - 一旦服务器故障，整个服务可能中断。
- **应用:** 网站（浏览器 - Web 服务器）、电子邮件系统。

P2P (Peer-to-Peer) 结构

- **定义:** 对等网络模式，网络中的每个节点（Peer）既是客户端也是服务器，节点之间可以直接进行通信和资源共享。

- **优点:**
 - 去中心化，不存在单点故障。
 - 扩展性好，网络规模越大，资源越丰富。
 - 能有效利用网络中闲置的计算和带宽资源。
- **缺点:**
 - 管理和控制困难，难以保证服务质量。
 - 安全性较低，节点间直接通信存在风险。
 - 性能不稳定，受限于参与节点的能力。
- **应用:** 文件共享（如 BitTorrent）、即时通讯软件。

四、CDN 相关知识

CDN 的作用

- **CDN (Content Delivery Network)** 即内容分发网络，通过在全球范围内部署大量边缘节点服务器，将网站的静态内容（如图片、视频、CSS、JS 文件）缓存到离用户最近的节点上，从而加速内容的分发。

CDN 工作原理

1. **用户请求:** 用户访问网站。
2. **DNS 智能解析:** DNS 服务器根据用户的地理位置，将请求解析到离用户最近的 CDN 边缘节点。
3. **缓存命中:** 如果边缘节点缓存了用户请求的内容，则直接将内容返回给用户。
4. **缓存未命中:** 如果边缘节点没有缓存该内容，则会从源服务器获取内容，并将其缓存到边缘节点，然后再返回给用户。

CDN 的优势

- **提升访问速度:** 就近获取内容，显著降低网络延迟。
- **减轻源服务器压力:** 大量用户请求由边缘节点处理，减少了回源请求。
- **提高网站可用性:** 分担源服务器流量，有效抵御 DDoS 攻击。
- **降低带宽成本:** 减少了长距离传输的数据量。

五、常见应用层协议

协议名称	英文全称	主要用途
HTTP	HyperText Transfer Protocol	万维网服务，传输超文本。
HTTPS	HTTP Secure	安全的 HTTP 协议，基于 SSL/TLS。
FTP	File Transfer Protocol	文件传输协议，用于在网络上传输文件。
SMTP	Simple Mail Transfer Protocol	简单邮件传输协议，用于发送电子邮件。
POP3	Post Office Protocol 3	邮局协议版本 3，用于接收电子邮件。
IMAP	Internet Message Access Protocol	互联网消息访问协议，用于接收和管理邮件。
SSH	Secure Shell	安全外壳协议，用于远程登录和文件传输。
Telnet	Teletype Network	远程登录协议，明文传输，安全性低。
DNS	Domain Name System	域名解析服务。

六、Socket 编程 (UDP 与 TCP)

Socket 简介

- Socket（套接字）是应用层与传输层之间的编程接口，是网络通信的基石。它为应用程序提供了一种发送和接收网络数据的机制。

TCP Socket 编程

- **特点:** 基于 TCP 协议，提供可靠的、面向连接的字节流服务。
- **服务器端流程:**
 - a. `socket()` : 创建一个 TCP 套接字。
 - b. `bind()` : 将套接字绑定到一个特定的 IP 地址和端口号。
 - c. `listen()` : 开始监听来自客户端的连接请求。
 - d. `accept()` : 接受客户端的连接，创建一个新的套接字用于与该客户端通信。
 - e. `recv()` / `send()` : 通过新套接字与客户端进行数据收发。
 - f. `close()` : 通信结束后，关闭套接字。
- **客户端流程:**
 - a. `socket()` : 创建一个 TCP 套接字。
 - b. `connect()` : 主动连接到服务器的 IP 地址和端口号。
 - c. `send()` / `recv()` : 与服务器进行数据收发。
 - d. `close()` : 通信结束后，关闭套接字。

UDP Socket 编程

- **特点:** 基于 UDP 协议，提供不可靠的、无连接的数据报服务。
- **服务器端流程:**
 - a. `socket()` : 创建一个 UDP 套接字。
 - b. `bind()` : 将套接字绑定到一个特定的 IP 地址和端口号。
 - c. `recvfrom()` : 从任意客户端接收数据报，并记录发送方的地址。
 - d. `sendto()` : 根据记录的地址，向客户端发送响应数据报。
 - e. `close()` : 关闭套接字。
- **客户端流程:**
 - a. `socket()` : 创建一个 UDP 套接字。
 - b. `sendto()` : 向服务器的 IP 地址和端口号发送数据报。
 - c. `recvfrom()` : 接收来自服务器的响应数据报。
 - d. `close()` : 关闭套接字。

步骤	TCP (可靠管道)	UDP (快速投递)
1. 准备	Server 必须 bind + listen。	Server 必须 bind。Client 啥都不用干。
2. 握手	Client 调 connect 触发三次握手。 Server 调 accept 被唤醒。	无。直接干。
3. 状态	全双工。双方 socket 绑定死，不用再传地址。	无状态。每次发送都要指定 (IP, Port)。
4. 接收	recv() 收到的是流，可能要自己拼包。	recvfrom() 收到的是完整包，附带发送者地址。
5. 结束	双方 close(), 触发四次挥手。	随时 close(), 不通知对方。